

AFL AGENT — MONITORING CHECKLIST

Week 6 Day 5 | Production Monitoring & Maintenance Reference

Purpose. One-page reference for what to monitor in production, when to investigate, and when to trigger model refresh or retraining. Source of truth: logs/agent_events.jsonl, with one JSON record per turn (implemented through src/logging_utils.py).

1. Response Latency

Metric	How to compute	Alert threshold
p50 / p95 / p99 turn latency	Use total_latency_ms per record, windowed hourly.	p95 > 8s → investigate; p95 > 15s → urgent response.
Tool latency	Monitor tools_called[].latency_ms.	Any call > TOOL_TIMEOUT_SECONDS (10s) is cut off and logged as status="timeout".
LLM fallback frequency	Count model_used != PRIMARY_MODEL per hour.	>20% fallback turns → provider/proxy may be degraded.

2. Tool Error Rate

Metric	How to compute	Alert threshold
Tool error rate	status="tool_error" / total turns.	>5% sustained over 1 hour → investigate.
Per-tool error rate	Group tools_called[].status="error" by tool name.	One tool consistently failing → check its CSV/model file and loadability.
Agent crash rate	Count status="agent_error".	Any nonzero rate → same-day investigation.

3. Off-Topic Leak Rate

Definition. Fraction of clearly off-topic or injection-style messages that receive a substantive answer instead of an AFL redirect.

Metric	How to compute	Alert threshold
Leak rate	Run the prompt_injection + scope subset of evaluation/day5_test_cases.py weekly against the live model; calculate % that fail to refuse.	>0% on prompt-injection cases → priority fix.
Heuristic-only catch rate	Measure guardrails.looks_like_override_attempt() on the same set.	A drop indicates new injection phrasing; update detection patterns.

4. Prediction Accuracy Drift

Metric	How to compute	Alert threshold
Match-winner accuracy vs. baseline	Re-run the holdout comparison in evaluation/day5_evaluation_report.md on newly completed rounds.	If accuracy falls within ~5 points of the h2h/form baseline → retraining candidate.
Distribution shift	Compare current-season feature averages (form, h2h win rate) with training-data distribution.	Large shift → flag for retraining review.

5. Alert Thresholds Summary

Signal	Warning	Page / Urgent
p95 latency	> 8s	> 15s
Tool error rate	> 5% / hr	> 15% / hr
Agent crash rate	> 0.5% / hr	> 2% / hr
Injection leak rate	> 0% in weekly eval	Repeated same-case failure across 2 weekly runs
Rate-limit hits	Informational only	Sudden ~10× baseline → possible bot/abuse traffic

6. Evaluation / Retraining Cadence

Weekly: Re-run evaluation/day5_test_cases.py (all 30 cases) against the live model. Track category pass rates. Re-run model-vs-baseline accuracy as new AFL rounds are appended to match_prediction_features.csv.

Per round: Refresh data/*.csv with newly completed round results. Recompute last-3 average, last-5 average, and h2h win-rate features. Keep rolling features current to avoid stale prediction inputs.

Retraining trigger: Retrain match-winner / top-player models if accuracy drops within 5 points of baseline for 2 consecutive weekly evaluations OR a full season of new data has accumulated since the last training run.

7. Weekly Data / Model Refresh Loop

Step	Process	Output / Decision
1	New round results published	Start refresh cycle
2	Append results to cleaned_round_by_round_stats_v2.csv and cleaned_team_matches.csv	Updated source data
3	Regenerate match_prediction_features.csv and player_prediction_features.csv using .shift(1) leakage guard	Current prediction features
4	Run the 30-case evaluation + model-vs-baseline accuracy comparison	Performance evidence
5	If retraining trigger is met → retrain, re-validate on	Updated model, if trigger met

	holdout, redeploy joblib files	
6	Log results in evaluation/ for trend tracking	Historical trend record